WEEK-10

1.PROBLEM STATEMENT

Design an **Online Shopping System** using Java collections.

The system should:

1. Maintain a list of products using a **linear collection (ArrayList)**.
2. Quickly search for a product by its ID using a **non-linear collection (HashMap)**.
3. Process customer orders according to their priority using a **PriorityQueue**.
4. Display the products, search result, and orders processed according to priority.

SOURCE CODE:

```
import java.util.*;
class Product {
   int id;
   String name;
   double price;
   Product(int id, String name, double price) {
      this.id = id;
      this.name = name;
      this.price = price;
   }
   public String toString() {
      return id + " " + name + " " + price;
   }
}
class Order implements Comparable<Order> {
```

```
    int orderId;
    int productId;
    int priority;
    Order(int orderId, int productId, int priority) {
        this.orderId = orderId;
        this.productId = productId;
        this.priority = priority;
    }
    @Override
    public int compareTo(Order o) {
        return Integer.compare(this.priority, o.priority);
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Linear collection - ArrayList
        ArrayList<Product> products = new ArrayList<>();
        // Non-linear collection - HashMap
        HashMap<Integer, Product> productMap = new HashMap<>();
        // Priority Queue for orders
        PriorityQueue<Order> orders = new PriorityQueue<>();
        int n = sc.nextInt();
        // Add products
        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
```

```
    String name = sc.next();
    double price = sc.nextDouble();
    Product p = new Product(id, name, price);
    products.add(p);
    productMap.put(id, p);
}
// Search product
int searchId = sc.nextInt();
// Add orders
int m = sc.nextInt();
for (int i = 0; i < m; i++) {
    int orderId = sc.nextInt();
    int productId = sc.nextInt();
    int priority = sc.nextInt();
    orders.add(new Order(orderId, productId, priority));
}
System.out.println("Products:");
for (Product p : products) {
    System.out.println(p);
}
// Search using HashMap
System.out.println("Search Result:");
Product result = productMap.get(searchId);
if (result != null) {
    System.out.println(result);
} else {
```

```
            System.out.println("Product Not Found");
        }
        // Process orders by priority
        System.out.println("Orders by Priority:")
        while (!orders.isEmpty()) {
            Order o = orders.poll();
            System.out.println(
                "Order " + o.orderId +
                " Product " + o.productId +
                " Priority " + o.priority
            );
        }
        sc.close();
    }
}
```

OUTPUT:

Plain text

```
4
101 Laptop 55000
102 Mobile 25000
103 Headphones 3000
104 Keyboard 1500
103
3
501 101 2
502 103 1
503 102 3
```

Sample Output

Plain text

```
Products:
101 Laptop 55000.0
102 Mobile 25000.0
103 Headphones 3000.0
104 Keyboard 1500.0
Search Result:
103 Headphones 3000.0
Orders by Priority:
Order 502 Product 103 Priority 1
Order 501 Product 101 Priority 2
Order 503 Product 102 Priority 3
```

2.PROBLEM STATEMENT:

A teacher wants to create a Java program to store the marks of students in a class. The number of students is **not known in advance**, so the program should use a **dynamic array**.

The teacher can:

- Add marks one by one.
- Automatically increase the array size when it becomes full.
- Display all the marks entered.

SOURCE CODE:

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Initial array size
        int[] marks = new int[2];
        int count = 0
        System.out.println("Enter marks (-1 to stop):");
        while (true) {
            int mark = sc.nextInt();
            // Stop entering marks
            if (mark == -1) {
                break;
            }
            // If array is full, increase its size
            if (count == marks.length) {
```

```
            int[] newMarks = new int[marks.length * 2];
            // Copy old elements
            for (int i = 0; i < marks.length; i++) {
                newMarks[i] = marks[i];
            }
            marks = newMarks;
        }
        // Add new mark
        marks[count] = mark;
        count++;
    }
    System.out.println("Student Marks:");
    for (int i = 0; i < count; i++) {
        System.out.println(marks[i]);
    }
    sc.close();
  }
}
```

OUTPUT:

Sample Input

```
85
72
90
68
95
-1
```

Sample Output

```
Enter marks (-1 to stop):
Student Marks:
85
72
90
68
95
```

3.PROBLEM STATEMENT:

A web browser stores recently visited web pages. When the user presses the **Back** button, the **most recently visited page should be removed first**.

Design a Java program using a **Stack** to:

- Visit web pages and store them in browser history.
- Display the current history.
- Perform the Back operation.
- Remove the most recently visited page first.

SOURCE CODE:

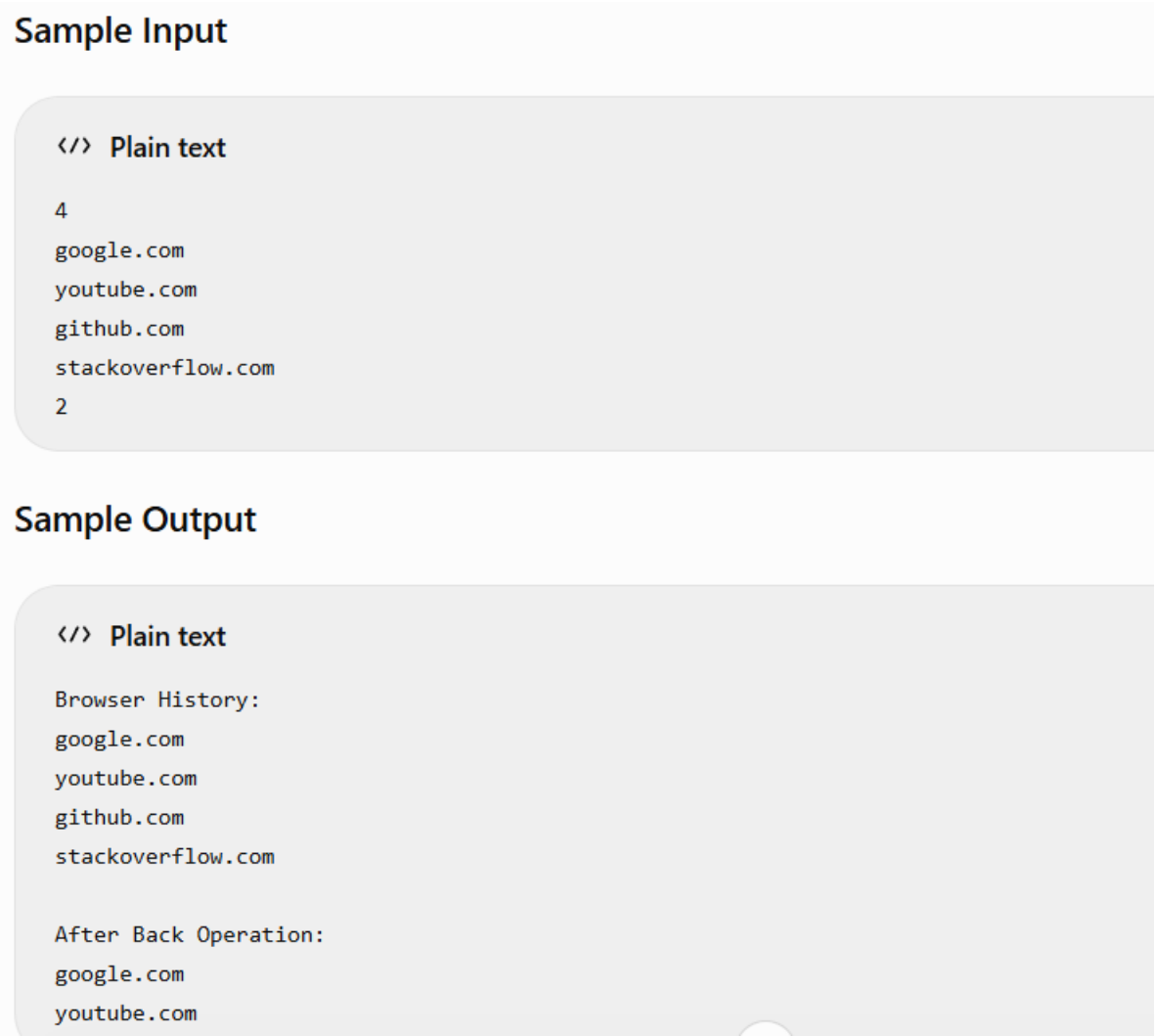
```
import java.util.Scanner;

import java.util.Stack;
```

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Stack<String> history = new Stack<>();
        int n = sc.nextInt();
        sc.nextLine();
        // Store visited pages
        for (int i = 0; i < n; i++) {
            String page = sc.nextLine();
            history.push(page);
        }
        System.out.println("Browser History:");
        for (String page : history) {
            System.out.println(page);
        }
        // Number of Back operations
        int back = sc.nextInt();
        System.out.println("After Back Operation:");
        for (int i = 0; i < back; i++) {
            if (!history.isEmpty()) {
                history.pop();
            }
        }
        for (String page : history) {
            System.out.println(page);
```

```
        }
        sc.close();
    }
}
```

OUTPUT:



4.PROBLEM STATEMENT:

A teacher does not know the number of students in advance. Student marks need to be stored dynamically.

Write a Java program using an **ArrayList** to:

- Store marks.
- Display marks.
- Find the highest mark.
- Calculate the average.

SOURCE CODE:

```
import java.util.ArrayList;
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Integer> marks = new ArrayList<>();
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            marks.add(sc.nextInt());
        }
        int highest = marks.get(0);
        int sum = 0;
        System.out.println("Student Marks:");
        for (int mark : marks) {
            System.out.println(mark);
            if (mark > highest) {
                highest = mark;
            }
            sum += mark;
        }
```

```
        double average = (double) sum / marks.size();
        System.out.println("Highest Mark: " + highest);
        System.out.println("Average Mark: " + average);
        sc.close();
    }
}
```

OUTPUT:

```
Enter number of students:
5
Enter marks:
85
72
90
68
95
Student Marks:
85
72
90
68
95
Highest Mark: 95
Average Mark: 82.0
```

5.PROBLEM STATEMENT:

A music application maintains songs in a playlist. Users can add songs to the playlist and remove a song when they no longer want it.

Implement a **singly linked list playlist** that supports:

1. Adding songs.
2. Removing a song.

3. Displaying the playlist.

SOURCE CODE:

```
import java.util.Scanner;

class Node {
    String song;
    Node next;

    Node(String song) {
        this.song = song;
        this.next = null;
    }
}

class Playlist {
    Node head;

    void addSong(String song) {
        Node newNode = new Node(song);

        if (head == null) {
            head = newNode;
        } else {
            Node temp = head;
```

```
        while (temp.next != null) {
            temp = temp.next;
        }

        temp.next = newNode;
    }
}

void removeSong(String song) {
    if (head == null) {
        return;
    }

    if (head.song.equals(song)) {
        head = head.next;
        return;
    }

    Node temp = head;

    while (temp.next != null) {
        if (temp.next.song.equals(song)) {
            temp.next = temp.next.next;
            return;
        }
```

```
            temp = temp.next;
        }
    }

    void display() {
        Node temp = head;

        while (temp != null) {
            System.out.println(temp.song);
            temp = temp.next;
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Playlist playlist = new Playlist();

        System.out.println("Enter number of songs:");
        int n = sc.nextInt();
        sc.nextLine();

        System.out.println("Enter songs:");
```

```
        for (int i = 0; i < n; i++) {
            playlist.addSong(sc.nextLine());
        }

        System.out.println("Enter song to remove:");
        String remove = sc.nextLine();

        playlist.removeSong(remove);

        System.out.println("Playlist:");
        playlist.display();

        sc.close();
    }
}
```

OUTPUT:

```
Enter number of songs:
4
Enter songs:
Believer
Perfect
ShapeOfYou
Faded
Enter song to remove:
Perfect
Playlist:
Believer
ShapeOfYou
Faded
```

6.PROBLEM STATEMENT:

A customer purchases several items, including repeated items. Write a Java program using a **LinkedHashSet** to display only unique items in the order they were purchased.

SOURCE CODE:

```
import java.util.LinkedHashSet;
import java.util.Scanner;


public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);


        LinkedHashSet<String> items = new LinkedHashSet<>();


        System.out.println("Enter number of items:");
        int n = sc.nextInt();
        sc.nextLine();
        System.out.println("Enter items:");
        for (int i = 0; i < n; i++) {
            items.add(sc.nextLine());
        }
        System.out.println("Unique Items:");
        for (String item : items) {
            System.out.println(item);
        }


        sc.close();
    }
```

}

OUTPUT:

```
Enter number of items:
7
Enter items:
Laptop
Mobile
Headphones
Mobile
Mouse
Laptop
Keyboard
Unique Items:
Laptop
Mobile
Headphones
Mouse
Keyboard
```

7.PROBLEM STATEMENT:

A teacher enters students' marks, but some marks are repeated. Write a Java program using a **TreeSet** to display the unique marks in **ascending order**.

SOURCE CODE:

```
import java.util.Scanner;

import java.util.TreeSet;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        TreeSet<Integer> marks = new TreeSet<>();

        System.out.println("Enter number of students:");

        int n = sc.nextInt();

        System.out.println("Enter marks:");

        for (int i = 0; i < n; i++) {

            marks.add(sc.nextInt());
```

```
        }
        System.out.println("Unique Marks in Ascending Order:");
        for (int mark : marks) {
            System.out.println(mark);
        }
        sc.close();
    }
}
```

OUTPUT:

```
Enter number of students:
7
Enter marks:
85
72
90
85
68
90
95
Unique Marks in Ascending Order:
68
72
85
90
95
```

8.PROBLEM STATEMENT:

Two students have selected different subjects. Write a Java program using the retainAll() method to find the subjects common to both students.

SOURCE CODE:

```
import java.util.HashSet;
import java.util.Scanner;
```

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        HashSet<String> student1 = new HashSet<>();
        HashSet<String> student2 = new HashSet<>();

        System.out.println("Enter number of subjects for Student 1:");
        int n1 = sc.nextInt();
        sc.nextLine();

        System.out.println("Enter subjects:");
        for (int i = 0; i < n1; i++) {
            student1.add(sc.nextLine());
        }

        System.out.println("Enter number of subjects for Student 2:");
        int n2 = sc.nextInt();
        sc.nextLine();

        System.out.println("Enter subjects:");
        for (int i = 0; i < n2; i++) {
            student2.add(sc.nextLine());
        }
```

```
        student1.retainAll(student2);

        System.out.println("Common Subjects:");

        for (String subject : student1) {
            System.out.println(subject);
        }

        sc.close();
    }
}
```

OUTPUT:

```
Enter number of subjects for Student 1:
4
Enter subjects:
Java
Python
DBMS
Maths
Enter number of subjects for Student 2:
4
Enter subjects:
Python
Java
OS
Networks
Common Subjects:
Java
Python
```

9.PROBLEM STATEMENT:

A teacher wants to store the marks of students using their **roll numbers as keys**. Write a Java program using a **HashMap** to display the marks of a particular student.

SOURCE CODE:

```
import java.util.HashMap;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        HashMap<Integer, Integer> marks = new HashMap<>();

        System.out.println("Enter number of students:");
        int n = sc.nextInt();

        System.out.println("Enter roll number and marks:");

        for (int i = 0; i < n; i++) {
            int rollNo = sc.nextInt();
            int mark = sc.nextInt();

            marks.put(rollNo, mark);
        }

        System.out.println("Enter roll number to search:");
        int rollNo = sc.nextInt();

        if (marks.containsKey(rollNo)) {
            System.out.println("Marks: " + marks.get(rollNo));
```

```
        } else {
            System.out.println("Student Not Found");
        }

        sc.close();
    }
}
```

OUTPUT:

```
Enter number of students:
4
Enter roll number and marks:
101 85
102 72
103 90
104 68
Enter roll number to search:
103
Marks: 90
```

10.PROBLEM STATEMENT:

A mobile application stores people's **names and phone numbers**. Users can search for a contact or update an existing phone number.

Write a Java program using a **HashMap** to:

1. Store contact names and phone numbers.
2. Search for a contact.
3. Update an existing phone number.
4. Display the updated contact details.

SOURCE CODE:

```
import java.util.HashMap;
```

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        HashMap<String, String> contacts = new HashMap<>();

        System.out.println("Enter number of contacts:");
        int n = sc.nextInt();
        sc.nextLine();

        System.out.println("Enter name and phone number:");

        for (int i = 0; i < n; i++) {
            String name = sc.nextLine();
            String phone = sc.nextLine();

            contacts.put(name, phone);
        }

        System.out.println("Enter contact name to search:");
        String searchName = sc.nextLine();

        if (contacts.containsKey(searchName)) {
            System.out.println("Phone Number: " + contacts.get(searchName));
```

```
        } else {
            System.out.println("Contact Not Found");
        }

        System.out.println("Enter contact name to update:");
        String updateName = sc.nextLine();

        if (contacts.containsKey(updateName)) {
            System.out.println("Enter new phone number:");
            String newPhone = sc.nextLine();

            contacts.put(updateName, newPhone);

            System.out.println("Contact Updated Successfully");
            System.out.println("Phone Number: " + contacts.get(updateName));
        } else {
            System.out.println("Contact Not Found");
        }

        sc.close();
    }
}
```

OUTPUT:

```
Enter number of contacts:
3
Enter name and phone number:
Surendra
9876543210
Rahul
9123456780
Priya
9988776655
Enter contact name to search:
Rahul
Phone Number: 9123456780
Enter contact name to update:
Rahul
Enter new phone number:
9000011111
Contact Updated Successfully
Phone Number: 9000011111
```

Week 11

1.PROBLEM STATEMENT :

Given an array nums of size n, return the majority element. The majority element appears more than ⌊n/2⌋ times. A majority element is guaranteed to exist.

Java Code

```
import java.util.*;


public class Result {


    public static int majorityElement(List<Integer> nums) {
        HashMap<Integer, Integer> map = new HashMap<>();


        for (int x : nums) {
            map.put(x, map.getOrDefault(x, 0) + 1);
        }


        for (int x : nums) {
            if (map.get(x) > nums.size() / 2) {
                return x;
            }
        }


        return -1;
    }
}
```

Sample Input

3

3 2 3

Sample Output

3

2 .PROBLEM STATEMENT :

Given a string s, find the first non-repeating character and return its index. If it does not exist, return -1.

SOURCE CODE

```
class Solution {
    public int firstUniqChar(String s) {

        int[] count = new int[26];

        for (char c : s.toCharArray()) {
            count[c - 'a']++;
        }

        for (int i = 0; i < s.length(); i++) {
            if (count[s.charAt(i) - 'a'] == 1) {
                return i;
            }
        }

        return -1;
```

```
    }
}
```

Sample Input

loveleetcode

Sample Output

2

3.PROBLEM STATEMENT :

Given a non-empty array of non-negative integers, the degree of the array is the maximum frequency of any element. Find the smallest possible length of a contiguous subarray having the same degree as the original array.

SOURCE CODE

```
import java.util.*;


public class Result {


    public static int degreeOfArray(List<Integer> nums) {


        HashMap<Integer, Integer> count = new HashMap<>();
        HashMap<Integer, Integer> first = new HashMap<>();
        HashMap<Integer, Integer> last = new HashMap<>();


        for (int i = 0; i < nums.size(); i++) {
            int x = nums.get(i);
```

```
            count.put(x, count.getOrDefault(x, 0) + 1);

            if (!first.containsKey(x)) {
                first.put(x, i);
            }

            last.put(x, i);
        }

        int degree = 0;

        for (int x : count.keySet()) {
            degree = Math.max(degree, count.get(x));
        }

        int answer = nums.size();

        for (int x : count.keySet()) {
            if (count.get(x) == degree) {
                int length = last.get(x) - first.get(x) + 1;
                answer = Math.min(answer, length);
            }
        }

        return answer;
    }
```

}

Sample Input

6

1 2 2 3 1 4

Sample Output

2

4.PROBLEM STATEMENT :

Given the names and grades of students, store them in a nested list and print the name(s) of the student(s) having the second lowest grade. If multiple students have that grade, print their names alphabetically.

SOURCE CODE

```
import java.util.*;


public class Main {


    public static void main(String[] args) {


        Scanner sc = new Scanner(System.in);


        int n = sc.nextInt();


        ArrayList<ArrayList<Object>> students = new ArrayList<>();


        for (int i = 0; i < n; i++) {


            String name = sc.next();
```

```
    double grade = sc.nextDouble();

    ArrayList<Object> student = new ArrayList<>();

    student.add(name);
    student.add(grade);

    students.add(student);
}

ArrayList<Double> grades = new ArrayList<>();

for (ArrayList<Object> student : students) {
    grades.add((Double) student.get(1));
}

Collections.sort(grades);

double lowest = grades.get(0);
double secondLowest = 0;

for (double grade : grades) {
    if (grade > lowest) {
        secondLowest = grade;
        break;
    }
```

```
        }

        ArrayList<String> names = new ArrayList<>();

        for (ArrayList<Object> student : students) {

            if ((Double) student.get(1) == secondLowest) {
                names.add((String) student.get(0));
            }
        }

        Collections.sort(names);

        for (String name : names) {
            System.out.println(name);
        }
    }
}
```

Sample Input

5

Harry 37.21

Berry 37.21

Tina 37.2

Akriti 41

Harsh 39

Sample Output

Berry

Harry

5.PROBLEM STATEMENT :

You are given several lines containing integers. For each query, find the number at position y of line x. If the position does not exist, print ERROR!.

SOURCE CODE

```
import java.util.*;


public class Main {


    public static void main(String[] args) {


        Scanner sc = new Scanner(System.in);


        int n = sc.nextInt();


        ArrayList<ArrayList<Integer>> list = new ArrayList<>();


        for (int i = 0; i < n; i++) {


            int size = sc.nextInt();


            ArrayList<Integer> row = new ArrayList<>();
```

```
            for (int j = 0; j < size; j++) {
                row.add(sc.nextInt());
            }

            list.add(row);
        }

        int q = sc.nextInt();

        for (int i = 0; i < q; i++) {

            int x = sc.nextInt();
            int y = sc.nextInt();

            if (x <= list.size() && y <= list.get(x - 1).size()) {
                System.out.println(list.get(x - 1).get(y - 1));
            } else {
                System.out.println("ERROR!");
            }
        }
    }
}
```

Sample Input

```
2
3 1 5 4
5 2 3 6 7 8
```

3

1 2

2 4

1 5

Sample Output

5

7

ERROR!

6.PROBLEM STATEMENT :

Given an integer numRows, return the first numRows of Pascal's Triangle. Each number is the sum of the two numbers directly above it.

SOURCE CODE

```
import java.util.*;


class Solution {


    public List<List<Integer>> generate(int numRows) {


        List<List<Integer>> result = new ArrayList<>();


        for (int i = 0; i < numRows; i++) {


            List<Integer> row = new ArrayList<>();


            for (int j = 0; j <= i; j++) {
```

```
            if (j == 0 || j == i) {
                row.add(1);
            } else {
                row.add(
                    result.get(i - 1).get(j - 1)
                    + result.get(i - 1).get(j)
                );
            }
        }

        result.add(row);
    }

    return result;
  }
}
```

Sample Input

5

Sample Output

[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]]

7.PROBLEM STATEMENT :

Given the root of a binary tree, print the values using **preorder traversal**. Preorder traversal visits:

Root → Left → Right

SOURCE CODE

```
static void preOrder(Node root) {
```

```
    if (root == null)
        return;

    System.out.print(root.data + " ");

    preOrder(root.left);

    preOrder(root.right);
}
```

Sample Tree

```
      1
     / \
    2   3
   / \
  4   5
```

Sample Output

1 2 4 5 3

The document does not provide a specific sample input for this question.

8.PROBLEM STATEMENT :

Given an array of integers and a target value, find two numbers whose sum equals the target. Return their indices.

SOURCE CODE

```
import java.util.*;

class Solution {
```

```
    public int[] twoSum(int[] nums, int target) {

        HashMap<Integer, Integer> map = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {

            int need = target - nums[i];

            if (map.containsKey(need)) {
                return new int[]{map.get(need), i};
            }

            map.put(nums[i], i);
        }

        return new int[]{};
    }
}
```

Sample Input

nums = [2,7,11,15]

target = 9

Sample Output

[0,1]

9.PROBLEM STATEMENT :

Given an array of bird sightings, find the bird type that occurs most frequently. If more than one type has the maximum frequency, return the smallest bird type ID.

SOURCE CODE

```
import java.util.*;


class Result {


    public static int migratoryBirds(List<Integer> arr) {


        int[] count = new int[6];


        for (int bird : arr) {
            count[bird]++;
        }


        int max = 0;
        int answer = 0;


        for (int i = 1; i <= 5; i++) {


            if (count[i] > max) {
                max = count[i];
                answer = i;
            }
        }
```

```
        return answer;
    }
}
```

Sample Input

6

1 4 4 4 5 3

Sample Output

4

10.PROBLEM STATEMENT:

A city has N junctions and M roads. Each road connects two junctions. Using a graph representation with collections, determine whether a route exists between the source and destination junctions. Print YES if a route exists; otherwise print NO.

SOURCE CODE

```
import java.util.*;


public class Main {


    public static void main(String[] args) {


        Scanner sc = new Scanner(System.in);


        int n = sc.nextInt();
        int m = sc.nextInt();
```

```
ArrayList<ArrayList<Integer>> graph = new ArrayList<>();

for (int i = 0; i <= n; i++) {
    graph.add(new ArrayList<>());
}

for (int i = 0; i < m; i++) {

    int u = sc.nextInt();
    int v = sc.nextInt();

    graph.get(u).add(v);
    graph.get(v).add(u);
}

int source = sc.nextInt();
int destination = sc.nextInt();

boolean[] visited = new boolean[n + 1];

Queue<Integer> queue = new LinkedList<>();

queue.add(source);
visited[source] = true;

while (!queue.isEmpty()) {
```

```
            int current = queue.poll();

            if (current == destination) {
                System.out.println("YES");
                return;
            }

            for (int next : graph.get(current)) {

                if (!visited[next]) {
                    visited[next] = true;
                    queue.add(next);
                }
            }
        }

        System.out.println("NO");
    }
}
```

Sample Input

```
5 4
1 2
2 3
3 4
4 5
```

1 5

Sample Output

YES

WEEK-12

1.PROBLEM STATEMENT:

A social media platform receives N hashtags. Count the frequency of each hashtag and display the hashtags along with their frequencies.

If multiple hashtags have the same frequency, display them in lexicographical order.

SOURCE CODE:

```
import java.util.*;


public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);


        // Read number of hashtags
        int n = sc.nextInt();


        // HashMap to store hashtag frequencies
        HashMap<String, Integer> frequency = new HashMap<>();
```

```
        // Read hashtags
        for (int i = 0; i < n; i++) {
            String hashtag = sc.next();

            frequency.put(hashtag, frequency.getOrDefault(hashtag, 0) + 1);
        }

        // Convert HashMap entries to a list
        List<Map.Entry<String, Integer>> list =
                new ArrayList<>(frequency.entrySet());

        // Sort by frequency descending,
        // and lexicographically if frequencies are equal
        Collections.sort(list, (a, b) -> {
            if (!a.getValue().equals(b.getValue())) {
                return b.getValue() - a.getValue();
            }
            return a.getKey().compareTo(b.getKey());
        });

        // Display result
        for (Map.Entry<String, Integer> entry : list) {
            System.out.println(entry.getKey() + " " + entry.getValue());
        }

        sc.close();
    }
}
```

Sample Input:

8

#java #python #java #coding #python #java #coding #html

Sample Output:

#java 3

#coding 2

#python 2

#html 1

2.PROBLEM STATEMENT:

You are given a collection of strings followed by a collection of query strings. For every query, determine how many times it occurs in the original collection.

For large input, the solution should avoid scanning the entire collection for every query.

SOURCE CODE:

```
import java.util.*;


public class Main {


    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);


        int n = sc.nextInt();


        HashMap<String, Integer> map = new HashMap<>();


        // Store string frequencies
        for (int i = 0; i < n; i++) {
```

```
            String str = sc.next();

            map.put(str, map.getOrDefault(str, 0) + 1);
        }

        int q = sc.nextInt();

        // Process queries
        for (int i = 0; i < q; i++) {
            String query = sc.next();

            System.out.println(map.getOrDefault(query, 0));
        }

        sc.close();
    }
}
```

Sample Input

```
4
aba
baba
aba
xzxb
3
aba
xzxb
ab
```

Sample Output

```
2
```

1

0

3.PROBLEM STATEMENT:

Given an integer array and an integer K, find the K most frequently occurring elements.

SOURCE CODE:

```
import java.util.*;


class Solution {
    public int[] topKFrequent(int[] nums, int k) {

        // Count frequencies
        HashMap<Integer, Integer> map = new HashMap<>();

        for (int num : nums) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        // Min-heap based on frequency
        PriorityQueue<Integer> pq = new PriorityQueue<>(
            (a, b) -> map.get(a) - map.get(b)
        );
```

```
        // Keep only k elements
        for (int num : map.keySet()) {
            pq.add(num);

            if (pq.size() > k) {
                pq.poll();
            }
        }

        // Store result
        int[] result = new int[k];

        for (int i = 0; i < k; i++) {
            result[i] = pq.poll();
        }

        return result;
    }
}
```

Input:

nums =

[1,1,1,2,2,3]

k =

2

Output:

[2,1]

4.PROBLEM STATEMENT:

Given a list of strings, group strings that are anagrams of one another.

Two strings are anagrams if they contain the same characters with the same frequencies.

SOURCE CODE:

```
import java.util.*;


class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {

        HashMap<String, List<String>> map = new HashMap<>();

        for (String str : strs) {

            // Convert string to character array
            char[] chars = str.toCharArray();

            // Sort characters
            Arrays.sort(chars);

            // Sorted string as key
            String key = new String(chars);

            // Add string to corresponding group
            map.putIfAbsent(key, new ArrayList<>());
            map.get(key).add(str);
        }

        return new ArrayList<>(map.values());
    }
```

}

Input:

strs =

[“eat”,”tea”,”tan”,”ate”,”nat”,”bat”]

Output:

[["eat","tea","ate"],["bat"],["tan","nat"]]

5.PROBLEM STATEMENT:

Maintain a collection of integers and support three operations:

1 x → Insert x

2 x → Delete one occurrence of x

3 f → Check whether any number occurs exactly f times

Print 1 if an element has frequency f, otherwise print 0.

SOURCE CODE:

```
import java.util.*;

class Result {

    public static List<Integer> freqQuery(List<List<Integer>> queries) {

        HashMap<Integer, Integer> freq = new HashMap<>();
        HashMap<Integer, Integer> freqCount = new HashMap<>();

        List<Integer> result = new ArrayList<>();

        for (List<Integer> query : queries) {

            int type = query.get(0);
            int value = query.get(1);
```

```
if (type == 1) {

    int oldFreq = freq.getOrDefault(value, 0);
    int newFreq = oldFreq + 1;

    freq.put(value, newFreq);

    if (oldFreq > 0) {
        freqCount.put(oldFreq,
            freqCount.get(oldFreq) - 1);
    }

    freqCount.put(newFreq,
        freqCount.getOrDefault(newFreq, 0) + 1);

} else if (type == 2) {

    int oldFreq = freq.getOrDefault(value, 0);

    if (oldFreq > 0) {

        int newFreq = oldFreq - 1;

        freq.put(value, newFreq);

        freqCount.put(oldFreq,
            freqCount.get(oldFreq) - 1);
```

```
                if (newFreq > 0) {
                    freqCount.put(newFreq,
                        freqCount.getOrDefault(newFreq, 0) + 1);
                }
            }

        } else if (type == 3) {

            if (freqCount.getOrDefault(value, 0) > 0) {
                result.add(1);
            } else {
                result.add(0);
            }
        }
    }

    return result;
  }
}
```

Sample Input

```
8
1 5
1 6
3 2
1 10
1 10
1 6
2 5
3 2
```

Sample Output

0

1

6.PROBLEM STATEMENT:

Given an array and a window size K, find the maximum number of distinct elements present in any contiguous window of size K.

SOURCE CODE:

```
import java.util.*;


public class Main {
    public static void main(String[] args) {


        Scanner sc = new Scanner(System.in);


        int n = sc.nextInt();
        int[] arr = new int[n];


        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }


        int k = sc.nextInt();


        HashMap<Integer, Integer> map = new HashMap<>();


        // First window
        for (int i = 0; i < k; i++) {
            map.put(arr[i], map.getOrDefault(arr[i], 0) + 1);
```

```
    }

    int maxDistinct = map.size();

    // Slide the window
    for (int i = k; i < n; i++) {

        // Remove outgoing element
        int outgoing = arr[i - k];

        map.put(outgoing, map.get(outgoing) - 1);

        if (map.get(outgoing) == 0) {
            map.remove(outgoing);
        }

        // Add incoming element
        int incoming = arr[i];

        map.put(incoming, map.getOrDefault(incoming, 0) + 1);

        // Update maximum
        maxDistinct = Math.max(maxDistinct, map.size());
    }

    System.out.println(maxDistinct);

    sc.close();
}
```

}

Input:

8

1 2 1 3 4 3 2 1

Output:

4

7.PROBLEM STATEMENT:

A city has N junctions and M roads. Each road connects two junctions.

Using a nested collection, determine whether a route exists from a source junction to a destination junction.

SOURCE CODE:

```
import java.util.*;


public class Main {


    public static boolean routeExists(
            ArrayList<ArrayList<Integer>> graph,
            int source,
            int destination) {


        boolean[] visited = new boolean[graph.size()];


        Queue<Integer> queue = new LinkedList<>();


        queue.add(source);
        visited[source] = true;
```

```
    while (!queue.isEmpty()) {

        int current = queue.poll();

        if (current == destination) {
            return true;
        }

        for (int next : graph.get(current)) {

            if (!visited[next]) {
                visited[next] = true;
                queue.add(next);
            }
        }
    }

    return false;
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();
    int m = sc.nextInt();

    ArrayList<ArrayList<Integer>> graph =
            new ArrayList<>();
```

```
        // Create nested collection
        for (int i = 0; i <= n; i++) {
            graph.add(new ArrayList<>());
        }

        // Read roads
        for (int i = 0; i < m; i++) {

            int u = sc.nextInt();
            int v = sc.nextInt();

            graph.get(u).add(v);
            graph.get(v).add(u);
        }

        // Source and destination
        int source = sc.nextInt();
        int destination = sc.nextInt();

        if (routeExists(graph, source, destination)) {
            System.out.println("YES");
        } else {
            System.out.println("NO");
        }

        sc.close();
    }
}
```

Sample Input

6 5

1 2

2 3

3 4

4 5

5 6

1 6

Output

YES

8.PROBLEM STATEMENT:

Given N words and an integer K, find the K most frequent words.

Rules:

1. Higher frequency comes first.
2. If two words have the same frequency, the lexicographically smaller word comes first.

SOURCE CODE:

```
import java.util.*;


public class Main {


    public static List<String> topKFrequent(String[] words, int k) {


        // Count frequency of each word

        HashMap<String, Integer> map = new HashMap<>();


        for (String word : words) {

            map.put(word, map.getOrDefault(word, 0) + 1);
```

```
}

// Min-heap
PriorityQueue<String> pq = new PriorityQueue<>(
    (a, b) -> {
        int freqA = map.get(a);
        int freqB = map.get(b);

        if (freqA != freqB) {
            return freqA - freqB;
        }

        // Lexicographically larger word has lower priority
        return b.compareTo(a);
    }
);

// Keep only k most frequent words
for (String word : map.keySet()) {

    pq.offer(word);

    if (pq.size() > k) {
        pq.poll();
    }
}

// Build result
List<String> result = new ArrayList<>();
```

```
        while (!pq.isEmpty()) {
            result.add(pq.poll());
        }

        // Reverse because min-heap gives lowest priority first
        Collections.reverse(result);

        return result;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        String[] words = new String[n];

        for (int i = 0; i < n; i++) {
            words[i] = sc.next();
        }

        int k = sc.nextInt();

        List<String> result = topKFrequent(words, k);

        System.out.println(result);

        sc.close();
```

```
    }
}
```

Sample Input

6

i love leetcode i love coding

2

Sample Output

[i, love]

9.PROBLEM STATEMENT:

A university stores student records containing:

Student ID

Student Name

Marks

Given multiple queries containing a student ID, efficiently retrieve the corresponding student's name and marks.

The solution must avoid scanning all student records for each query.

SOURCE CODE:

```
import java.util.*;


class Student {
    String name;
    int marks;

    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }
```

```
}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Number of students
        int n = sc.nextInt();

        // Student ID -> Student record
        HashMap<Integer, Student> students = new HashMap<>();

        // Store student records
        for (int i = 0; i < n; i++) {

            int id = sc.nextInt();
            String name = sc.next();
            int marks = sc.nextInt();

            students.put(id, new Student(name, marks));
        }

        // Number of queries
        int q = sc.nextInt();

        // Process queries
        for (int i = 0; i < q; i++) {
```

```
            int id = sc.nextInt();

            Student student = students.get(id);

            if (student != null) {
                System.out.println(
                    student.name + " " + student.marks
                );
            } else {
                System.out.println("Student not found");
            }
        }

        sc.close();
    }
}
```

Sample Input

4

101 Ravi 85

102 Priya 92

103 Arun 78

104 Sneha 95

3

102

104

105

Sample Output

Priya 92

Sneha 95

Student not found

10.PROBLEM STATEMENT:

A supermarket maintains product information:

Product ID

Product Name

Stock

Price

Perform the following:

1. Update stock after sales.
2. Find products whose stock is below a given threshold.
3. Display products sorted by remaining stock.
4. Efficiently retrieve products using their ID.

SOURCE CODE:

```
import java.util.*;


class Product {
    int id;
    String name;
    int stock;
    double price;

    Product(int id, String name, int stock, double price) {
        this.id = id;
        this.name = name;
        this.stock = stock;
        this.price = price;
```

```
    }

    public String toString() {
        return id + " " + name + " " + stock + " " + price;
    }
}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        HashMap<Integer, Product> inventory = new HashMap<>();

        // Number of products
        int n = sc.nextInt();

        // Store products
        for (int i = 0; i < n; i++) {

            int id = sc.nextInt();
            String name = sc.next();
            int stock = sc.nextInt();
            double price = sc.nextDouble();

            inventory.put(id,
                new Product(id, name, stock, price));
        }
```

```
// Update stock after sale
int productId = sc.nextInt();
int quantitySold = sc.nextInt();

Product soldProduct = inventory.get(productId);

if (soldProduct != null) {
    if (soldProduct.stock >= quantitySold) {
        soldProduct.stock -= quantitySold;
    }
}

// Low-stock threshold
int threshold = sc.nextInt();

System.out.println("Low Stock Products:");

for (Product p : inventory.values()) {
    if (p.stock < threshold) {
        System.out.println(p);
    }
}

// Sort products by remaining stock
List<Product> products =
        new ArrayList<>(inventory.values());

products.sort((a, b) -> a.stock - b.stock);
```

```
        System.out.println("Products Sorted by Stock:");

        for (Product p : products) {
            System.out.println(p);
        }

        // Efficient retrieval by ID
        int searchId = sc.nextInt();

        Product found = inventory.get(searchId);

        System.out.println("Product Search:");

        if (found != null) {
            System.out.println(found);
        } else {
            System.out.println("Product not found");
        }

        sc.close();
    }
}
```

Sample Input

```
4
101 Rice 50 60.0
102 Sugar 20 45.0
103 Oil 10 120.0
104 Salt 30 25.0
```

101 45

10

103

Sample Output

Low Stock Products:

103 Oil 10 120.0

Products Sorted by Stock:

101 Rice 5 60.0

103 Oil 10 120.0

102 Sugar 20 45.0

104 Salt 30 25.0

Product Search:

103 Oil 10 120.0